

Space Odyssey

A Space Tycoon Idle/Progression Game for Discord

Sachin Shankar • Tesla STEM High School, Redmond, WA • sachinshankar@live.com •
github.com/shankar-sachin/space-odyssey

OVERVIEW

Space Odyssey is a fully-featured idle and progression game built as a Discord bot using **Discord.js v14** and **SQLite**. Players build rockets, construct and manage space stations, sign contracts with real-world space agencies, research new technologies, explore and colonize destinations across the solar system, and grow a space company from the ground up — all inside Discord via slash commands, interactive buttons, and modal forms. The game features passive income systems, a tiered research tree, canvas-rendered visual overviews, dynamic contract generation, and a crew management system with health and entertainment mechanics.

ARCHITECTURE & ENGINEERING

Space Odyssey is structured around a strict layer separation that keeps game logic decoupled from Discord's API surface, making logic independently testable and maintainable.

Stack	Node.js, Discord.js v14, SQLite (via <code>better-sqlite3</code>), Canvas (PNG rendering)
Interaction Flow	<code>index.js</code> routes all Discord events: slash commands → <code>commands/<name>.js</code> → <code>execute()</code> ; button clicks routed by prefix (<code>rkt_</code> , <code>launch_</code> , <code>contract_</code> , etc.); modal submissions routed by custom ID
Game Layer	<code>game/</code> directory holds all pure business logic (rockets, stations, contracts, economy, research, planets, events) with zero Discord imports — fully decoupled
Commands	<code>commands/</code> handles Discord interaction surface; calls into <code>game/</code> and <code>database/</code> only
Database	SQLite with WAL mode and foreign keys; 15 tables (<code>players</code> , <code>companies</code> , <code>rockets</code> , <code>stations</code> , <code>station_modules</code> , <code>station_energy</code> , <code>station_crew</code> , <code>contracts</code> , <code>research</code> , <code>planet_progress</code> , and more). Schema auto-migrates on startup.
Config	<code>config.js</code> is the single source of truth for all game balance: component costs, tiers, build times, contract templates, research trees, station revenue multipliers
Utils	Shared embed templates, button builders, Canvas PNG rendering (<code>utils/canvas.js</code>), and a global cooldown middleware (5-second cooldown on all non-admin commands, enforced pre-dispatch in <code>index.js</code>)

Stateful multi-step builder: `commands/build.js` uses an in-memory `Map` to track multi-step rocket and module construction across button presses. Each press advances the builder state (engine → fuel → hull → payload → summary → name modal → DB commit) without requiring a persistent session or re-prompting the user.

Canvas caching: 800×600 PNG images for empire, fleet, station, and solar system views are generated via `utils/canvas.js` and cached for 60 seconds to avoid redundant renders on frequent button presses.

GAME SYSTEMS

Rockets & Launches

- **Component tiers** — Engines, fuel tanks, hulls, and payloads each have tiered variants (basic → advanced → elite) with escalating costs and performance stats defined in `config.js`.
- **Launch success rate** — Determined at launch time: 30–95% based on component quality. Outcome rolls happen live; failed launches consume the rocket.
- **Range rating** — Each rocket has a `range_rating` derived from its components. Launching modules to non-Earth stations requires sufficient range to reach the station's destination.
- **Exploration probes** — Rockets can be sent as one-way probes to discover destinations. Success rate is reduced by `difficulty × 0.05`; on success, a `planet_progress` row is created with `status = 'discovered'`.

Space Stations

- **Construction** — Players place a station frame (Ω1,000) and choose a location — Earth Orbit (always available) or any discovered destination. Location is stored in `stations.planet`; launching modules to remote stations requires adequate rocket range.
- **Operational requirements** — A station is only *operational* (generating revenue) when Core+Docking, Comms, an active energy source, and crew are all present.
- **Module types** — Core/Docking, Communications, Storage, Habitat, Garden, Lab, Observation. Modules upgrade from Lv0 to Lv2 (Garden to Lv3); multipliers are cumulative: Lv1 = ×2.0, Lv2 = ×3.0, Garden Lv3 = ×9.0 base revenue.
- **Energy sources** — 7 types: Solar I–III, Nuclear I–II, Bio I, Bio SII. Each has a `supports` count (max module slots) and optional servicing/lifetime cycles. Nuclear II has a 4-day lifetime. Bio I/SII are free but require a Garden Module (Bio SII needs Garden at Lv3) and generate a +Ω40/hr bonus.
- **Passive income** — Time-based revenue accrues while the player is offline; `/collect` harvests credits using timestamp deltas on `last_collected`.

Crew System

- **Crew types** — Three tiers: **Lant** (Ω5k, 2-module cap), **Sauf** (Ω20k, 6-module cap), **Kirt** (Ω80k, unlimited). Crew is required for station operational status.
- **Health & entertainment** — Both stats decay over time. Dropping below 20% on either takes the station offline. Managed via `/crew care` or in-station buttons.

Contracts & Agencies

- **Dynamic generation** — Contracts are generated procedurally; difficulty 1–5 scales rewards and requirements (payload capacity, range, specific station modules). Acceptance is gated by player level and reputation.
- **Agency tiers** — 7 real-world agencies unlock at player level milestones: ISRO (Lv1), Roscosmos (Lv5), UAESA (Lv9), CNSA (Lv10), JAXA (Lv15), ESA (Lv18), NASA (Lv22).

Research & Technology

- **Six tech trees** — Unlock elite components and new gameplay features. Research nodes have prerequisite chains.
- **Colony tech** — Requires `deep_space_nav` + `station_efficiency`. Unlocks `/colonize` — allowing players to establish colonies on discovered non-gas-giant destinations that generate passive `hourlyIncome` collected via `/collect`.

Solar System & Destinations

- **15 destinations** — Three types: **planets** (Moon, Mars, Venus), **moons** (Titan, Europa, Ganymede, Callisto, Enceladus, Rhea, Dione, Mimas, Io — each with a **parent** field), and **gas giants** (Jupiter, Saturn, Neptune). Gas giants are `stationOnly` — they can be orbited by stations but not colonized.
- **Discovery gating** — Each destination has a `minRange` requirement; rockets must meet the threshold to attempt exploration.

THIRD-PARTY API INTEGRATION & IMAGE GENERATION

Space Odyssey integrates third-party image generation APIs to produce dynamic in-game visuals, giving players rendered feedback for their empire, fleet, station layouts, and solar system territory. Visual overviews are generated as **800×600 PNG images** via `utils/canvas.js` using the **Canvas API**, with a 60-second cache layer to prevent redundant renders. Images are delivered directly into Discord embeds, creating a polished visual experience. The territory canvas renders each station positioned near its declared planet, giving a live spatial view of the player's presence across the solar system.

SERVER WORKFLOWS & AUTOMATION

Beyond the core game, Space Odyssey includes a suite of server management and automation tools accessible to administrators:

- **Reaction roles** — Admins can create reaction-role button panels in any channel via `/admin react`, with message content configured through a modal. Users click buttons to self-assign roles.
- **Welcome messages** — Configurable welcome message system via `/admin greet` — post formatted welcome embeds in any channel with custom content.

- **Level-role tiers** — Five XP-based tiers (Member, Explorer, Pioneer, Veteran, Legend) can each be assigned a Discord role via `/admin level set`. Role assignments are automatic as players level up.
- **User management** — Admins can forcibly adjust any user's level and XP via `/admin level adjust`, view all configured tier assignments, and audit registered slash commands.
- **Game administration** — Full game admin suite: `godmode` (max out all player stats for testing), `spawn` (create ready rockets by component tier), `spawnmodule` (create ready station modules), `gencontracts` (force-generate contracts), and `danger wipe` (full data reset with confirmation gate).

CURRENT STATUS

Space Odyssey is actively developed and publicly available on GitHub. The core game loop — rocket construction, station building, contract completion, research progression, and solar system exploration — is fully implemented. Crew management, energy systems, module leveling, colony passive income, and canvas-rendered visual overviews are all live. Server workflow tools (reaction roles, welcome messages, level-role automation) are complete.

Repository github.com/shankar-sachin/space-odyssey

Stack Node.js, Discord.js v14, SQLite, Canvas

Status Active development — core systems complete, ongoing balance tuning and feature additions